

Java Full Stack Daily Notes

Class – 1 & 2

* Programming:

- Programming is writing instructions for a computer to perform specific tasks.
- Instructions are written in languages that computers can understand.
- To automate tasks, solve problems & create systems and applications, programming is used.

Types of programming languages:

1. High-Level Language: Closer to human language, easy to read and write.
ex: Java, Python etc.
2. Low-Level Language: Closer to machine language, harder to understand. ex: Assembly, Machine code. [Machine understandable language - binary language]

⇒ Translation Required: High-level code needs to be translated into low level for execution.

Languages of computers:

1. Binary System: Binary uses only two digits 0 and 1, representing off and on states.
2. Machine Language: All computer operations are ultimately expressed in binary.
3. Conversion: High-level code is translated to binary for computer execution.

* Compiles:

→ It translates the entire program into machine code before execution. [translates entire program at a time].

• Java $\xrightarrow{\text{Compiles}}$ Byte code $\xrightarrow{\text{JVM}}$ Binary code.
(.class)

* Interpreter:

→ It translates code line-by-line during execution.

* Java uses both Compiles & Interpreter.

→ It compiles to byte code then interpreted by JVM.

∴ It is known as hybrid language.

* WORA: Write Once Run Anywhere.

→ Java will not generate directly binary code. It will generate intermediate code (i.e., .class file) called byte code.

→ Java is a object oriented programming language.

History of Java.

→ James Gosling is the founder of the Java-programming language. He is known as the father of Java.

→ 1991 - Project Started: Java was initiated by Sun Microsystems as the Green Project and initial name was OAK.

→ 1995 - Java Released: Renamed it to Java and Java 1.0 launched with WORA (Write Once Run Anywhere) philosophy.

→ 2010 - Oracle Acquisition: Oracle acquired Sun Microsystems and took over Java.

→ 2014 - Java 8 Released: Some major changes happened in Java in this version.

→ 2025 - Java 24 Released: New Java Version released on March 18, 2025.

Features of Java.

class: 2

1. Platform Independent: Java bytecode can run on any device with JVM.

→ Operating Systems: Windows, Mac OS, Linux.

→ C language extension $\cdot c \rightarrow \text{compiler} \rightarrow \cdot \text{exe file (binary)}$.

→ These are dependent on OS.

→ Binary language is different in different OS i.e., in Windows, Mac OS, Linux.

→ It is a platform dependent language.

→ Disadvantage: We can't use one file in different platforms i.e., in different operating system (OS).

rate * HLL
(WORA)
(High Level Language)
• Java $\xrightarrow{\text{compiles}}$ Byte code $\xrightarrow{\text{JVM}}$ Binary.
(can't understand by Machine & person)

↓
This made java platform independent.

* In Java, Bytecode run in,

→ Windows ✓

→ Linux ✓

→ Mac OS ✓

↓
It works same in different OS [i.e., can understand this bytecode in any OS].

↓
This code run in any OS.

2. Object Oriented: Supports OOP (Object Oriented Programming) concepts like inheritance, encapsulation, & polymorphism.

Ex:

Class Test {

→ Class name & filename should be same.

}

[∵ Java is case sensitive language].

Ex:

```
class Test {  
    public static void main (String [] args) {  
        System.out.println ("Hello");  
    }  
}
```

→ main method.

output: Hello.

- Java will start only with main method.
- It will compile, but it will not execute without Main Method.
- Compile: ~~Java -c~~ javac filename.
- Run: Java filename

3. Object Strictly Typed Language:

- Requires explicit declaration of variable types, helping to catch type related errors at compile time.
- We should mention datatype in Java.

Ex:

int a = 10

String b = "Ravi"

boolean c = true

datatype

4. Automatic Garbage Collection:

- Manages memory automatically by removing unused objects.
- Unused variables collected by Garbage collection & it will trash.
- Trash: Memory free.

5. Robust:

- Java can handle errors gracefully and keeps running without crashing.
- Exception Handling.
- Gracefull shutdown.

6. Secure:

→ There are no pointers (C-language uses pointers), You can't directly access memory.

7. Backward Compatibility:

→ Java will support in any version but not viceversa.

Java 1.0 → Java 24.0 support.

d. → Python will not support in any version. It has no backward compatibility. Python 2 ≠ Python 3 ≠ Python 4.

→ Due to this feature, we can easily upgrade to new version in Java.

8. Deprecated:

→ Some methods not used in upgraded versions.

(like a warning/suggestion) → to upgrade versions of Java.

* Java is compiled and interpreted language which is done in two steps.

→ Java and JVM were designed with portability in mind.

→ The `javac` command-line tool compiles java source code into java class file containing platform-neutral bytecode.

→ The compiled class file (bytecode) can be executed by the

Java Virtual Machine (JVM).

* **JVM** - Java Virtual Machine (Not physical, it is virtual, invisible)

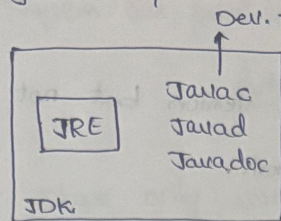
* **JDK** - Java Development Kit.

* **JRE** - Java Runtime Environment.

* JDK: Java Development Kit. (used by developers).

→ It includes JRE and other development tools.

→ Tools like Java Compiler (javac), javadoc generators.



→ $JDK = JRE + \text{Development tools}$
 Development tools
 Java compilers
 Java Debuggers (to understand mistakes).

JRE: Java Runtime Environment (After compilation, we have to run the code, to run that code we need JRE).

→ Developers everytime compile & run the code.

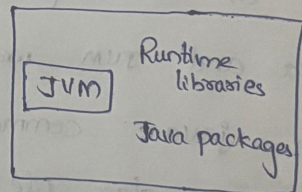
→ Clients will only run the code.

→ It is the implementation of JVM.

→ It includes java runtime libraries and .class file.

→ $JRE = JVM + \text{Run time libraries (java.util, java.io)}$
 input output
 default methods & classes
 System.out.println();

JVM: Java Virtual Machine.



→ It is called a virtual machine because it doesn't physically exist.

→ It will execute the java bytecode line by line.
 (.class file)

* Interpreter: Follow the instructions and run the code line by line.

→ JVM used it internally.

→ .class file (bytecode) executes line by line by interpreter.

→ It works only on .class file in Java.

→ JVM is responsible for Java platform Independent.

* Types Parts of JVM: 3 parts.

1. CLSS - Class Loader Sub System.
2. Memory Management.
3. Execution Engine.

* CLSS:

→ This loads .class file (bytecode) into the JVM, and also does verification - verifies vulnerabilities (damage / virus).

* Memory Management:

→ After loading the code, it assigns memory.
→ It has 5 parts.

1. Method Area:

→ Stores static variables, methods, class metadata (class information).

2. Heap Area:

→ Stores objects and instance variables / Non-static variables / Global variables.

3. Stack area:

→ Stores local variables, method call data (per thread) / threads (Any variable inside a method).

4. Program Counter Register:

→ Holds current and next instruction addresses.

5. Native Method Stack:

→ Executes native methods (eg: C, C++, python etc via JNI)
→ Responsible to support other languages.

Ex: `Static int c = 10` (Static Variable)

`int d = 20;` (Non-static / Instance / Global Variable).

`public void hi() { }` → Non-static method.

* Static: Constant / fixed - something which doesn't change.

→ Linked directly to the class, memory will assign only once

Ex:

class information
↑

Class Employee { → Method Area

String name = "Ravi" } → Heap memory - 4 bytes

int salary = 10; - 4 bytes

Static String Company = "FLM"; → Method area - 4 bytes

} (If we work in same company, company name will not change)

Objects to the class ↓
Heap area
{ Emp 1 - name, salary, company. - 12 bytes - memory required
Emp 2 - name, salary, company. - 12/8 bytes - "
Emp 3 - name, salary, company. - 12/8 bytes - "

[∵ Memory will assign only once to company name due to static string]

Ex:

class Test {

public static void main (String [] args) {

int a = 10; } → Local variables.

int b = 20;

}

* Execution Engine: It has 3 parts.

1. Interpreter: Executes byte code line by line (.class file).

2. JIT Compiler: Converts frequently used bytecode into native code for speed.

→ Executes in a single shot (not to repeat the same code by Interpreter i.e, repeated code).

→ Due to this, Java will become much faster.

3. Garbage Collector:

→ Frees memory by removing unused objects.

→ Trashes all the unused memory.